University of Gujrat

Assginment

Submitted By: Abdullah Khan

Roll No: [25011598-062]

Submitted To: Mr Muhammad Sabeeh.

Course Title: Data Structures

Course Code: DS - 202

Section: Data Science (A)

Submission Date:

CamScanner

Algorithm of Array

1) Initialize Set K := 1
   LOC = 1
   and Max := DATA[1]
2) Increment Counter Set K := K + 1
3) Test counter: if K > N
   write LOC, Max and exit
4) Compare and update:
   if MAX < DATA[K]
   set LOC := K and MAX := DATA[K]
5) Repeat Loop

Dry Run

→ Array: [5, 12, 9, 20, 7], N = 5
→ K = 1 : MAX = 5, LOC = 1
→ K = 2 : 5 < 12 is True → MAX = 12, LOC = 2
→ K = 3 : 12 < 9 is false
→ K = 4 : 12 < 20 is True → MAX = 20, LOC = 4
→ K = 5 : 20 < 7 is False.
→ K = 6 : 6 > 5 [True] Exit and output
LOC = 4, MAX = 20

Time complexity

O(N).

Insertion Sort Algorithm

1. Set A[0] := $-\infty$
2. Repeat step 3 to 5 for K > 1 to N
3. set temp := A[K] and PTR := K-1
4. Repeat while temp < A[PTR]
   → set A[PTR+1] := A[PTR]
   → set PTR := PTR - 1
5. Set [PTR+1] := temp
6. Exit

Dry Run

1. A = [15, 10, 5, 6, 8] with N = 5, A[0] = $-\infty$
2. Pass 1 (K=1): A[1]
3. Pass 2 (K=2):
   = temp = A[2] = 10, PTR = 1
   = Since temp < A[1] (10 < 15), shift 15 to A[2] making PTR 0
   = Sine temp > A[0] (10 > $-\infty$), so the while loop exits
   = Set A[1] = 10 sorted sub array (10, 15).
4. Pass 3 (K=3)
   = tem = A[3] = 5
   = 5 is compared and shifted past 15 and 10, ending up at A[1]
   Sorted sub array [5, 10, 15]

5. (k = 5):
Temp = A[5] = 8
8 is inserted at 3rd place
Final sorted [5, 6, 8, 10, 15].

Time complexity:
O(N)

Algorithm of Merge sort

1. start
2. Divide array AB into two subarrays A and B
3. set Mid = (N+1)/2
4. For I = 1 to Mid, set A[I] = AB[I]
5. For I = Mid + 1 to N, set B[I - Mid] = AB[I]
6. Sort subarrray A amd B
7. Merge two sorted Sub array into the main array

Dry Run:

A = [3, 8] B = [6, 13]

Compare A[1] (3) and B[1](6) 3 is small
Compare A[2](8) and B[1](6) 6 is small
Compare A[2](8) and B[2](13) 8 is Small
Finaly
[3, 6, 8, 13

Time complexity
O (N Log N)

② Array Traversal
start
Repeat step 3 for i = LB to UB
Print LA [i]

Dry run
Array [10, 20, 30], LB = 1 UB = 3
i = 1 Print 10
i = 2 print 20
i = 3 print 30

Time complexity
O(N)

③ 1-D insertion
1. Set j := N
2: Repeat step 3 and 4 while j >= k
Move jth element downward
3. set LA[j+1] := LA[j]
4. Decrease counter set j := j-1
5 insert set LA[k] := Item
6. Reset N: set N := N+1

Dry run
Array: [10, 20, 30], N = 3 k = 2.
Item = 99
shift Element index 3 down to 2:

Array [10, 20, 20, 30]
Insert 99 at ind 2
[10, 99, 20, 30], N=4

Time complexity
O(N)

(4) 1-Array Deletion
set ITem := LA[K]
Repeat for j = K to N-1
Move (J+1)st element
set [A[J] := LA[J+1]
Reset : set N := N-1

Dry run
Array [10, 20, 30], N=3 K=2
ITem = 20
Array beome [10, 30

Time complexity
O(N)

5. Array Searching
Insert ITem: Set DATA[N+1] := ITem
set K := 1
Repeat while DATA[K] ≠ Item
set K := K+1
if K = N+1

Dry run
Array [5, 10, 15], N=3, ITEM = 10

setinel added at index 4; [4] = 10
k = 1 (5 ≠ 10), then k = 2 (10 = 10)

Time complexity:
O(N)

6. 1-D Array Reversing

Repeat for i = 1 to Max/2 by 1
temp = arr[i]
arr[i] = arr(Max - i + 1),
arr[Max - i + 1] = temp

Dry run
Array: [1, 2, 3, 4], Max = 4
i = 1 swap[1] and become [4, 2, 3, 1]
i = 2 swap arr[2] & [3] Array become
[4, 3, 2, 1]

Time complexity
O(N)